Lab 5: Functions

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** Function definition, prototypes, parameters, return values, scope

Objectives

- Write and call user-defined functions; declare prototypes before `main`.
- Distinguish between parameters and arguments; understand pass-by-value.
- Decompose a problem into small, reusable functions.
- Reason about variable scope (local vs. global) and `return` semantics.

Quick Reference

Anatomy of a function.

```c
/* prototype (declaration) */
int  add(int a, int b);

int main(void) {
    int s = add(3, 4);   /* call: arguments 3 and 4 */
    return 0;
}

/* definition */
int add(int a, int b) {   /* parameters a, b */
    return a + b;          /* returns an int */
}
```

Void functions. Use `void` for no return and `void` parameter list for no inputs.
Pass-by-value. Each argument is *copied*. Changing a parameter inside the function does not affect the caller's variable. (Pointers, in Lab 9, let us change the caller's variable.)
Scope. A variable declared inside `{...}` exists only in that block.

Quick Experiments (Try It)

Try It 1: Pass-by-value: the caller's copy is *never* changed

Run this and record the output:

```c
#include <stdio.h>
void double_it(int x) {
    x = x * 2;
    printf("inside function: x = %d\n", x);
}
int main(void) {
    int a = 5;
    double_it(a);
    printf("back in main:    a = %d\n", a);
    return 0;
}
```

1. Was `a` changed? Why not?
2. Rewrite `double_it` to accept `int *x` and use `*x = (*x) * 2;`. Call it as `double_it(&a);` and verify `a` is now 10.

3. This is the core difference between Lab 5 (pass-by-value) and Lab 7 (pointers).

Try It 2: What if you call a function before its prototype?

Deliberately place the definition of `square` *after* `main` and remove any prototype:

```c
#include <stdio.h>
int main(void) {
    printf("%d\n", square(4));   /* called before defined */
    return 0;
}
int square(int x) { return x * x; }
```

1. Does it compile? What warning or error do you get?
2. Add the prototype `int square(int x);` before `main`. Does it compile now?
3. Why does the compiler need to see the prototype *before* the call?

Try It 3: Count recursive calls with a global counter

Add a global counter to your recursive `power` function and measure how many calls it makes:

```c
#include <stdio.h>
int calls = 0;
double power(double x, int n) {
    calls++;
    if (n == 0) return 1.0;
    return x * power(x, n - 1);
}
int main(void) {
    printf("%.0f\n", power(2.0, 10));
    printf("recursive calls: %d\n", calls);
    return 0;
}
```

How many calls are made for `power(2, 10)`? Now implement an iterative version with a loop – how many loop iterations does it take? Are the numbers the same? Why or why not?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: `max3` *(15 min)*

Write a function `int max3(int a, int b, int c)` that returns the largest of three integers. Test it from `main` on at least three different input triples.

In-Lab Exercise 2: Factorial and Binomial *(35 min)*

1. Write `long fact(int n)` returning $n!$ (iteratively).
2. Using `fact`, write `long C(int n, int k)` returning $\binom{n}{k} = n!/(k!(n-k)!)$.
3. In `main`, read n and k and print $C(n, k)$.

In-Lab Exercise 3: Reusable Prime Check *(30 min)*

Write `int is_prime(int n)` that returns 1 if `n` is prime, otherwise 0. In `main`, read two integers `a` and `b` and print every prime in $[a, b]$ on a single line, separated by spaces.

In-Lab Exercise 4: Recursive Power *(40 min)*

Write `double power(double x, int n)` for $n \geq 0$ using recursion: `power`$(x, 0) = 1$, otherwise `power`$(x, n) = x \cdot$ `power`$(x, n-1)$. Then write an *iterative* version `power_iter` and compare the outputs on at least three test cases.

AI Collaboration

Try these prompts with an AI tool:

- "In C, explain pass-by-value with a concrete example showing why changing a parameter inside a function does not affect the caller's variable."
- "What is a function prototype, and what happens if I omit it when the function is defined after `main`?"
- "Show me a recursive and an iterative version of factorial and compare how many operations each performs for $n = 10$."

Critical check – verify, don't just accept: Ask the AI to write a recursive function and ask how many recursive calls it makes for a specific input. Add a global call counter to the AI's code, run it, and verify the actual count. Then ask the AI whether the function risks a stack overflow for large n – test that claim by calling it with increasingly large arguments until you observe a crash or wrong result.

Know the limit: AI tools generate recursive functions without mentioning stack depth limits and rarely suggest an iterative alternative unless asked – always follow up with "is there a risk of stack overflow here?"

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Pass-by-value

What does this print?

```c
#include <stdio.h>
void f(int x) { x = x + 1; }
int main(void) {
    int a = 10;
    f(a);
    printf("%d\n", a);
    return 0;
}
```

Trace & Predict 2: Mutual recursion is unusual; here is plain recursion

What does `g(4)` return? Show every call.

```c
int g(int n) {
    if (n <= 1) return 1;
    return n * g(n - 1);
}
```

Find the Bug 1: Missing prototype, wrong return

Find and fix all problems:

```c
#include <stdio.h>
int main(void) {
    printf("%d\n", square(5));
    return 0;
}
square(int x) {
    x * x;
}
```

Write Code on Paper 1: Sum from a to b (inclusive)

Write a function `long sum_range(int a, int b)` that returns $a + (a+1) + \cdots + b$. Assume $a \leq b$. Then write a complete program that reads `a` and `b`, calls the function, and prints the result.

Write Code on Paper 2: Fibonacci function (iterative)

Write `long fib(int n)` that returns the n-th Fibonacci number iteratively ($F_0 = 0$, $F_1 = 1$). Then write `main` that prints $F_0, F_1, \ldots, F_{10}$, all on one line separated by spaces.